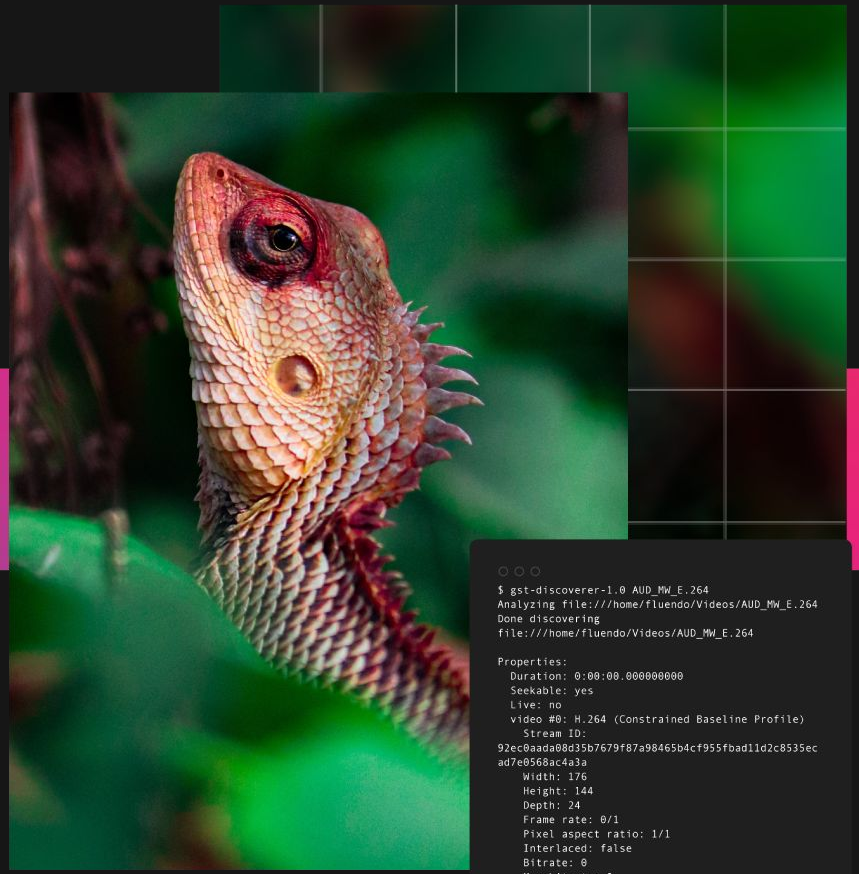
fluendo

Gst.WASM
Launched



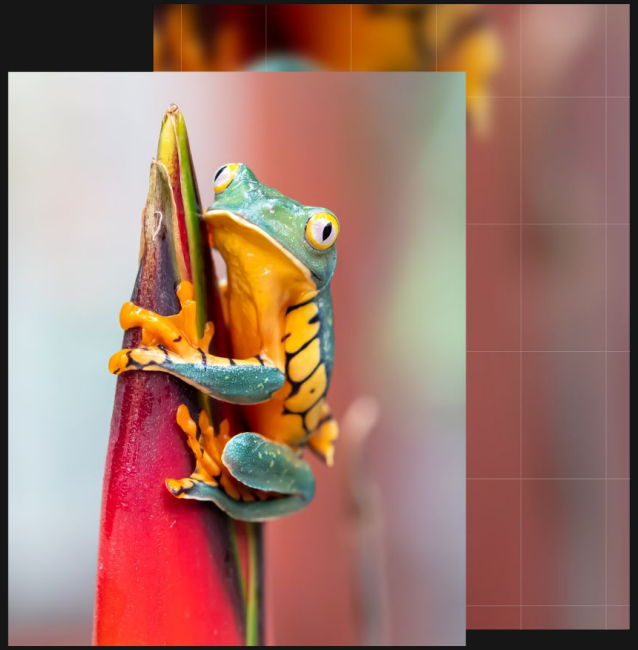
```
$ gst-discoverer-1.0 AUD_MW_E.264
Analyzing file:///home/fluendo/Videos/AUD_MW_E.264
Done discovering
file:///home/fluendo/Videos/AUD_MW_E.264

Properties:
  Duration: 0:00:00.000000000
  Seekable: yes
  Live: no
  video #0: H.264 (Constrained Baseline Profile)
    Stream ID:
92ec0aada08d35b7679f87a98465b4cf955fbad11d2c8535ec
ad7e0568ac4a3a
    Width: 176
    Height: 144
    Depth: 24
    Frame rate: 0/1
    Pixel aspect ratio: 1/1
    Interlaced: false
    Bitrate: 0
    Max bitrate: 0
```

Jorge Zapata 10/2024

Index

- Recap
- The project
- Design and Architecture
- Samples

Recap



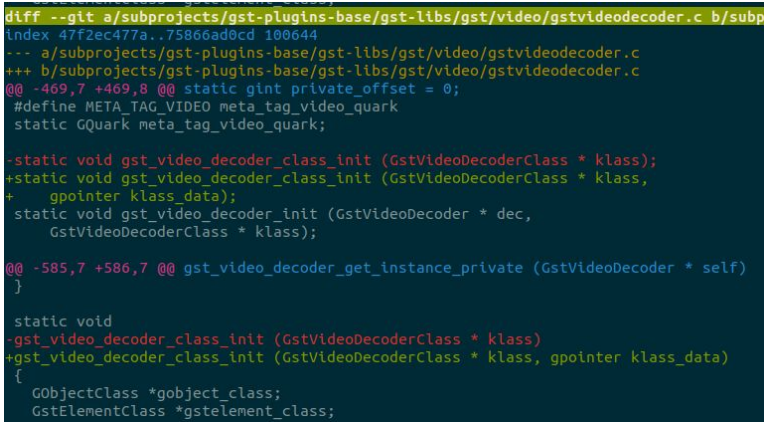
Overview

- We presented in 2023 an initial port of GStreamer (GLib, libffi, etc) to Emscripten (WebAssembly)
- Main goal was to improve the browser's codec offer by replacing the WebCodecs API with GStreamer

```
▶0:00:05.492255000    42    0x1a67d0 DEBUG          basetransform gstbasetransform.c:2174:default_generate_output:          test01.js:162
<glcolorbalance0> calling prepare buffer
▶0:00:05.492679000    42    0x1a67d0 DEBUG          basetransform gstbasetransform.c:1655:default_prepare_output_buffer:    test01.js:162
<glcolorbalance0> passthrough: reusing input buffer
▶0:00:05.492985000    42    0x1a67d0 DEBUG          basetransform gstbasetransform.c:2181:default_generate_output:          test01.js:162
<glcolorbalance0> using allocated buffer in 0x1ba6d0, out 0x1ba6d0
▶0:00:05.493235000    42    0x1a67d0 DEBUG          basetransform gstbasetransform.c:2192:default_generate_output:          test01.js:162
<glcolorbalance0> element is in passthrough
▶0:00:05.493479000    42    0x1a67d0 DEBUG          basetransform gstbasetransform.c:2383:gst_base_transform_chain:         test01.js:162
<glcolorbalance0> we have a pending DISCONT
▶0:00:05.493719000    42    0x1a67d0 TRACE     GST_REFCOUNTING gstobject.c:238:gst_object_ref:<sink:sink> 0x16bef8 ref    test01.js:162
1->2
▶0:00:05.493979000    42    0x1a67d0 TRACE     GST_REFCOUNTING gstobject.c:238:gst_object_ref:<sink> 0x16ba98 ref 6->7   test01.js:162
```

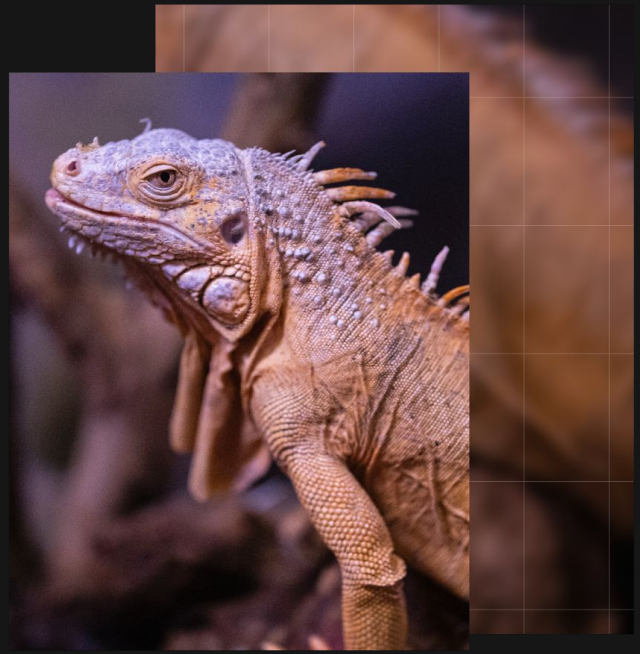
Migration and status

- Migration process had multiple issues
  - Full of function pointers mismatch
  - Lack of UNIX sockets
  - No video sink
  - Missing OpenGL (WebGL) integration
- No strategy to define next development steps
  - What functionalities are missing?
- How to contribute upstream?
  - Few branches available



```
diff --git a/subprojects/gst-plugins-base/gst-libs/gst/video/gstvideodecoder.c b/subp
index 47f2ec477a..75866ad0cd 100644
--- a/subprojects/gst-plugins-base/gst-libs/gst/video/gstvideodecoder.c
+++ b/subprojects/gst-plugins-base/gst-libs/gst/video/gstvideodecoder.c
@@ -469,7 +469,8 @@ static gint private_offset = 0;
 #define META_TAG_VIDEO meta_tag_video_quark
 static GQuark meta_tag_video_quark;

-static void gst_video_decoder_class_init (GstVideoDecoderClass * klass);
+static void gst_video_decoder_class_init (GstVideoDecoderClass * klass,
+    gpointer klass_data);
 static void gst_video_decoder_init (GstVideoDecoder * dec,
     GstVideoDecoderClass * klass);

@@ -585,7 +586,7 @@ gst_video_decoder_get_instance_private (GstVideoDecoder * self)
 }

 static void
-gst_video_decoder_class_init (GstVideoDecoderClass * klass)
+gst_video_decoder_class_init (GstVideoDecoderClass * klass, gpointer klass_data)
 {
   GObjectClass *gobject_class;
   GstElementClass *gstelement_class;
```

The project

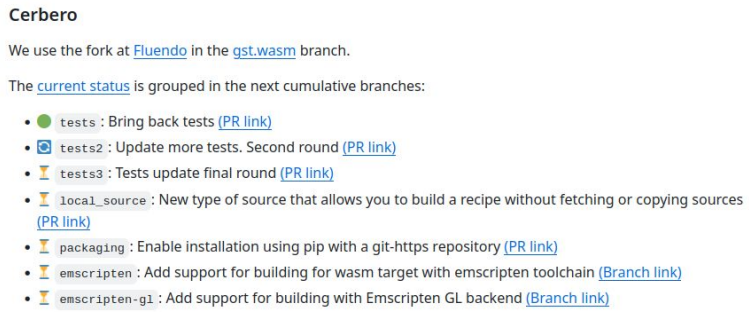


GitHub repository

- Publicly available at https://github.com/fluendo/gst.wasm
- Single place to
  - Keep track of *upstreamables* features (MRs to the community)
  - Have documentation focused on the actual integration between GStreamer and Emscripten
  - Have samples and tutorials
  - Have usable packages
  - Bug tracking

Upstreaming

- We use `guw` https://github.com/fluendo/git-upstream-workflow
- A rebase based upstream oriented workflow
- Keep all cumulative branches synced
- Allows to add, remove and sync features
- Easily integration of reviewed and merged MR
- Keep track of the status of each



Cerbero

We use the fork at Fluendo in the gst.wasm branch.

The current status is grouped in the next cumulative branches:

- `tests` : Bring back tests (PR link)
- `tests2` : Update more tests. Second round (PR link)
- `tests3` : Tests update final round (PR link)
- `local_source` : New type of source that allows you to build a recipe without fetching or copying sources (PR link)
- `packaging` : Enable installation using pip with a git-https repository (PR link)
- `emscripten` : Add support for building for wasm target with emscripten toolchain (Branch link)
- `emscripten-gl` : Add support for building with Emscripten GL backend (Branch link)

OpenGL support

- Part of the forked gst-plugins-base
- New `gl_platform`: emscripten, `gl_winsys`: canvas
- New `GstGLDisplayWeb`

GStreamer Web plugins

- `gst-plugins-web`
  - `/gst-libs/gst/web/` As a container for a common web library:
    - `webvideoframe` as specific GstMemory for Web VideoFrame
    - `webrunner` single thread dispatcher
    - `webcanvas` thread dispatcher per canvas and handled as a `GstContext`
  - `/ext/web/` For all web related elements:
    - `webcanvassrc` To gather <canvas> image content as a source element
    - `webcanvassink` To render image/video content into <canvas>
    - `webcodecsviddec[h264][sw,hw]` To decode using Web's WebCodecs API
    - `emhttpsrc` To fetch HTTP content through Web's Fetch API

Design and architecture

WebWorkers and pthreads

- A WebWorker allows running a script operation separate from the main execution thread of a web application
- WebWorkers communication mechanism is done by message passing, i.e `postMessage` and `onmessage`
- WebWorkers operate on *transferred objects* by posting a message
- WebWorkers can not access the DOM
- `pthreads` are implemented via a Web's WebWorker API
- Emscripten `pthreads` bootstrap code owns the message passing infra
- Posting custom messages for passing data among threads needs to be injected into Emscripten implementation

Transferable objects

- Not every object can be manipulated in a WebWorker, only transferable objects
- Transferable objects are:
  - ArrayBuffer
  - MessagePort
  - ReadableStream
  - WritableStream
  - TransformStream
  - WebTransportReceiveStream
  - WebTransportSendStream
  - AudioData
  - ImageBitmap
  - VideoFrame
  - OffscreenCanvas
  - RTCDataChannel

Transferable objects

- The only object Emscripten transfers to a pthread (WebWorker) are <canvas> elements
- Done at pthread creation with `g_thread_emscripten_new (name, canvas, func, data)`

Emscripten Embind and emscripten::val()

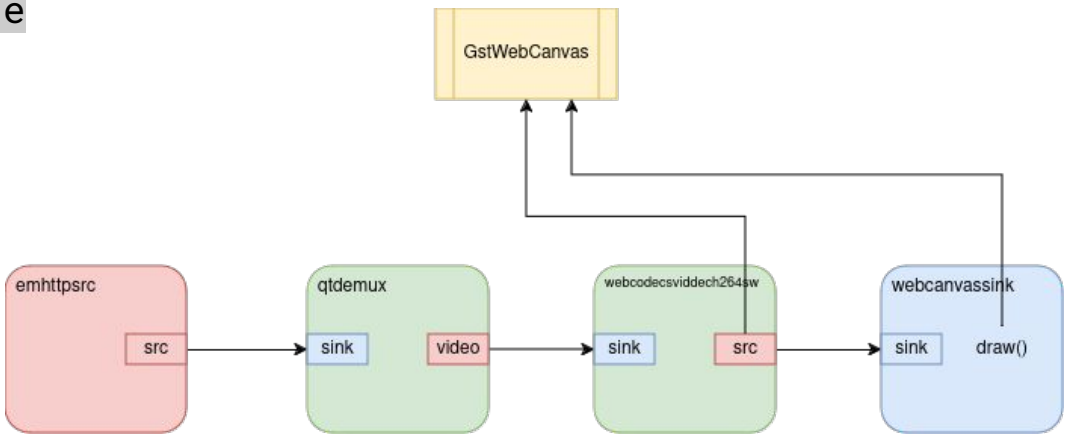
- `emscripten::val()` transliterates JS code to C++
- JavaScript values are per thread as each WebWorker has its own JavaScript environment

```
var xhr = new XMLHttpRequest;
xhr.open("GET", "http://url");
```

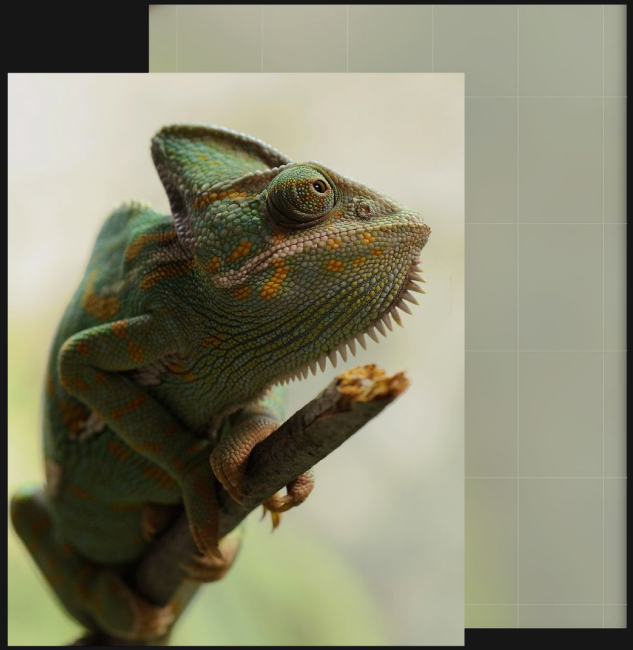
```
val xhr = val::global("XMLHttpRequest").new_();
xhr.call<void>("open", std::string("GET"), std::string("http://url"));
```

GstWebCanvas and GstWebRunner

- Seamless multi-threading support in GStreamer pipelines / elements
- Offload JavaScript access to Web features (VideoDecoder, OffscreenCanvas, Fetch, etc) into a single thread, similar to OpenGL implementation
- Transfer `emscripten::val()` among elements to maintain the flow
- Optimal workflow should transfer objects from one thread to the other with `postMessage`



Samples



Thank you!

Questions?